

系统辨识——利用Matlab自动调参

文档参考：

[华南小虎队：【技术与比赛】你还在瞎调PID吗？系统辨识用matlab，自动调参](#)

[同济大学SuperPower战队：系统辨识调PID全流程](#)

1. 前期工作

1.1 理解系统辨识

- 流程：

1. 给电机固定的扭矩（电流）（其他项目可以是PWM）输入
2. 利用**串口**获取想要调试的输出值（接下来以M3508的速度值为例进行介绍）
3. 到Matlab中进行**系统函数（传递函数）的拟合**（系统函数反映了电机输入和输出的关系）
4. 搭建simulink仿真
5. 手动调整响应速度和平稳状态，自动得到PID参数

- 注意事项：

1. 该方法适用于**“粗调”**，在不同的负载条件下快速获得**可用、合理的参数**。但非最完美的，后续还需要根据曲线情况和需要设定的目标值不同调整
2. 输出值的采样频率一般设置为**1kHz**，尽量能够模拟出最真实的数据变化状态，这样拟合的系统函数才足够精准

1.2 软件安装

1. [VOFA+:串口调试助手](#)

2. Matlab （主要使用simulink功能）

- 通过网盘分享的文件：MATLAB R2024b链接：

https://pan.baidu.com/s/15emQDOqxje02SxhK_cNKpg?pwd=tu6r 提取码: tu6r

1.3 代码仓库

包含simulink模型、csv、A板代码

[仓库](#)

2. 工作流

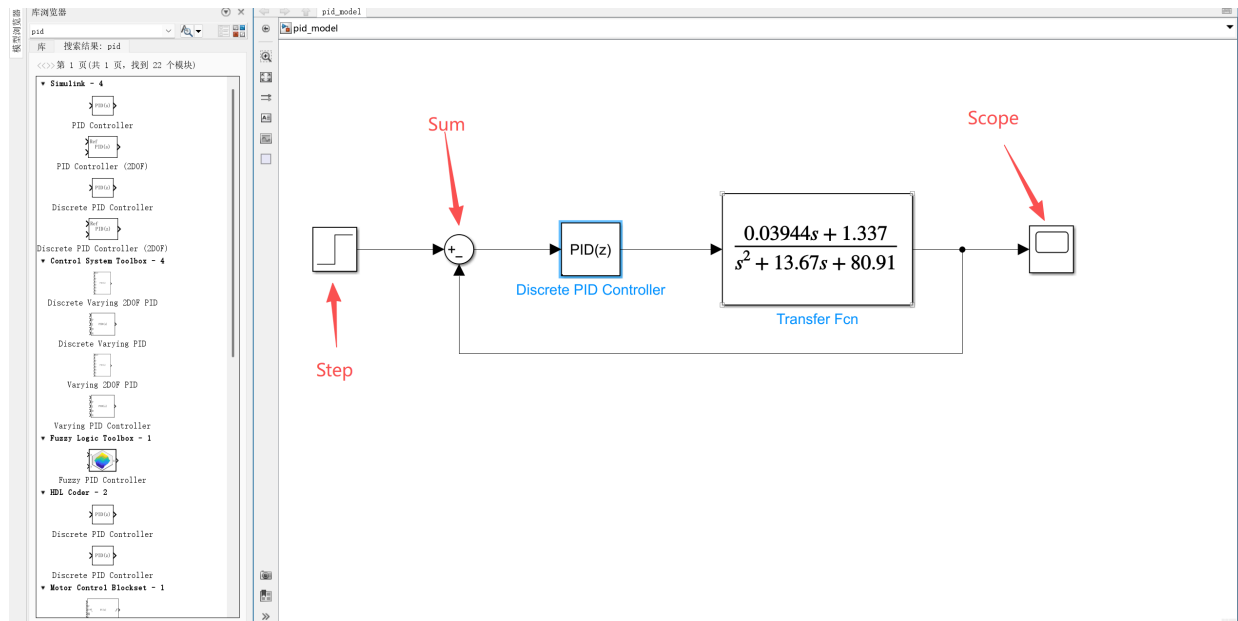
2.1 Simulink搭建

- 需要用到的模块

1. **Step**（输入的是阶跃函数）
2. **Sum**（加减法器，形成反馈的闭环）
3. **PID Controller**（离散、连续都行）
4. **Transfer Fcn**（传递函数）
5. **Scope**（示波器，查看输出波形）

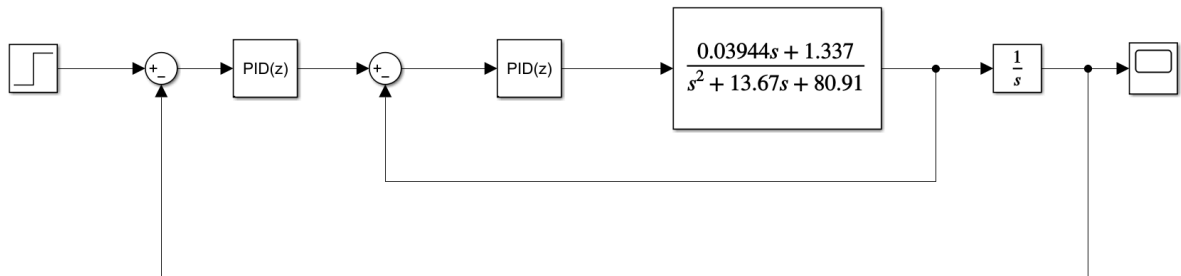
- 搭建效果如图（单环）

- 注意，这里的Step输入是速度，而输出才是扭矩
- 获取系统函数的时候是固定扭矩输入，得到速度的流程是为了得到系统函数



- 双环PID：

- Integrator（积分器）
- 多了一个 $1/s$ ，达到积分器的效果（信号与系统）。速度的积分是位置。
- 此时调参顺序为先调内环，再调外环



2.2 设置开环输出和串口打印

这是FreeRtos的任务接口

1. 电机初始化，模式选择开环输出

```
/**
 * @brief 初始化M3508电机对象及其PID
 */
void DJI_M3508_Task_Init(void)
{
    motor_3508.Init(&hcan1, CAN_Motor_ID_0x201, Control_Method_OPENLOOP);
}
```

2. Freertos任务接口

- 这里的采样频率为200Hz，可以改为更高
- 这里设定**固定扭矩为3000**，不至于转太慢，也不至于疯转。应根据不同的负载适当设置更低或更高
- 串口打印不仅打印了速度值，还打印了扭矩值，是为了观察什么时候输入扭矩使得电机速度开始变化

```
/**
 * @brief M3508电机的FreeRTOS任务入口函数
 * @param argument 任务参数（未使用）
 */
void DJI_M3508_Task(void *argument)
{
    int time_counter = 0;

    for(;;)
    {
        if ((time_counter > 1000) && (time_counter <= 10000))
        {
            motor_3508.Set_Target_Torque(3000.0f);
        }
        else
        {
            motor_3508.Set_Target_Torque(0.0f);
            if (time_counter > 10000)
            {
                time_counter = 0;
            }
        }

        uart_printf("speed: %f\r\n", motor_3508.Get_Now_Omega()); // 通过串口打
```

印当前速度

```
// 5ms采样一次
if (time_counter % 5 == 0)
{
    uart_printf("%f,%f,%f\r\n",motor_3508.Get_Now_Omega(),motor_3508.Get_Now_Torque(),motor_3508.Get_Target_Torque());// 通过串口打印当前速度
}

motor_3508.TIM_PID_PeriodElapsedCallback();

TIM_CAN_PeriodElapsedCallback();

time_counter++;

osDelay(1);
}
}
```

2.3 VOFA+串口采集数据

1. 模式选择：**FireWater**（这是根据代码仓库内容的设置，自己可以任意选择，但要注意串口发送的相关格式）
2. 查看数据能否正常打印
3. 保存数据



2. 数据格式为.csv，可以用Excel和Matlab打开



4. 查看数据并挑选

- 反馈的扭矩会出现不在设定的扭矩，且波动过大的情况，但由于输入电机的仍是设定值3000，**依旧认为输入扭矩是3000**
- 如下图，**A为速度值，B为反馈的扭矩。**
 1. 挑选一段合理的数据变化状态
 2. 数据段一般为400个以上

A1				✕	✓	j
	A	B				
5946	0	59				
5947	0	-47				
5948	0	26				
5949	0	-88				
5950	0	-15				
5951	0	59				
5952	0	26				
5953	0	-47				
5954	0	67				
5955	0	-88				
5956	0	-47				
5957	0	-55				
5958	0	-47				
5959	0	26				
5960	0	26				
5961	0	-88				
5962	0	-6				
5963	0.648936	2083				
5964	2.563024	2204				
5965	4.711602	2153				
5966	6.227603	2181				
5967	7.754511	2524				
5968	9.674053	2430				
5969	11.29367	2384				
5970	12.96236	1933				
5971	14.99096	2248				
5972	16.59967	2036				
5973	18.42651	2491				
5974	20.11701	2195				
5975	21.64392	2414				
5976	23.60709	2101				
5977	25.16126	1878				

5978	26.74815	2677	
5979	28.25325	1877	
5980	29.71472	2924	
5981	31.41068	2679	
5982	32.89941	2717	

2.4 系统辨别

2.4.1 导入数据

步骤如下：

1. **文件目录**设置为存放csv和slx（simulink模型）的文件夹
2. **双击**保存好数据的csv，出现如图导入工具
3. 拉取选择相应的数据，前面可以选择一部分0，后半段选择**平稳的数据**，如图后半段数据基本稳定于50左右
4. 导入

MATLAB R2025b

主窗体：文件、命令窗口、工作区、变量浏览器、脚本编辑器、Simulink、布局、设置、附加功能、并行、帮助、了解 MATLAB 资源。

文件浏览器：显示文件列表，包括 1.csv, 2.csv, motor3508, pid_model_double.slx, pid_model.slx, vofa+.csv。文件 2.csv 被选中。

导入工具：显示导入选项，包括文件类型、分隔符、所选内容/历史记录、名称、类型、将无法导入的单元格替换为 NaN。

导入数据预览：显示导入的数据预览，包括列名 A, B 和行号 10, 11。

导入数据表：显示导入的数据表，包括列名 A, B 和行号 6117, 6118, 6119, 6120, 6121, 6122, 6123, 6124, 6125, 6126, 6127, 6128, 6129。

导入数据表内容：

	A	B
10		11
数值	数值	
5957	0	-47
5958	0	26
5959	0	26
5960	0	-88
5961	0	-6
5962	0.64894	2083
5963	2.563	2204
5964	4.7116	2153
5965	6.2276	2181
5966	7.7545	2524
5967	9.6741	2430
5968	11.2937	2384
5969	12.0694	1023

导入数据表内容：

	A	B
6117	50.0608	1085
6118	50.0826	968
6119	50.1153	1057
6120	50.1971	1649
6121	50.1644	716
6122	50.0826	474
6123	50.0935	274
6124	50.0826	256
6125	50.1644	1256
6126	50.1262	892
6127	50.1644	942
6128	50.1044	980
6129	50.0935	435

2.4.2 设置好输入和输出

1. 转换Output数据

- 导入的数据是table格式，需要转换为数组array

▼ 工作区			
名称	值	大小	类
 output	<i>208×1 double</i>	208×1	double
 x2	<i>208×1 table</i>	208×1	table

- 执行命令行：output = table2array(x2)

命令行窗口

不熟悉 MATLAB? 请参阅[快速入门](#)的资源。

```
>> output = table2array(x2);
>>
```

- 2. 设置input

命令行窗口

不熟悉 MATLAB? 请参阅[快速入门](#)的资源。

```
>> output = table2array(x2);
>> input = [zeros(47,1);ones(161,1)]
```

- 由于Output前面选择了47个0，且认为Output非0时，电机才真正开始输入电流
- 所以input也设置为前面47个0
- 由于扭矩为3000，所以乘上3000

命令行窗口

不熟悉 MATLAB? 请参阅[快速入门](#)的资源。

1

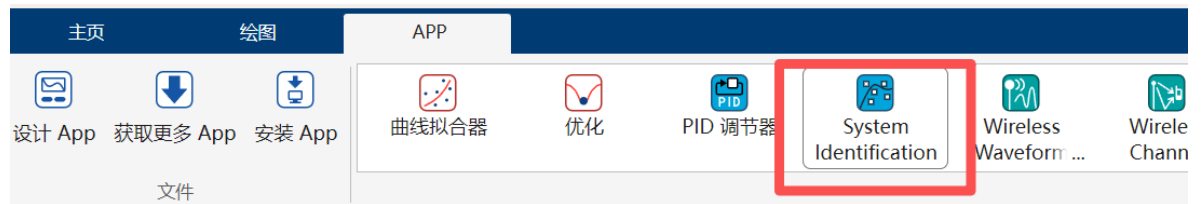
```
>> input = [zeros(47,1);ones(161,1)]*3000
```

input =

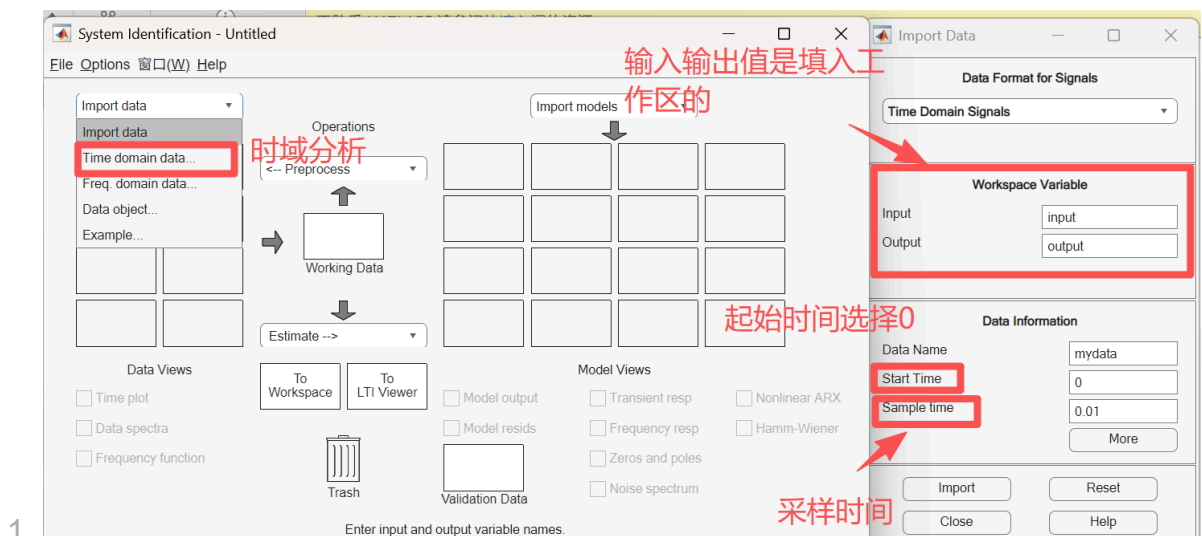
⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗ ⊗

3. 拟合系统函数

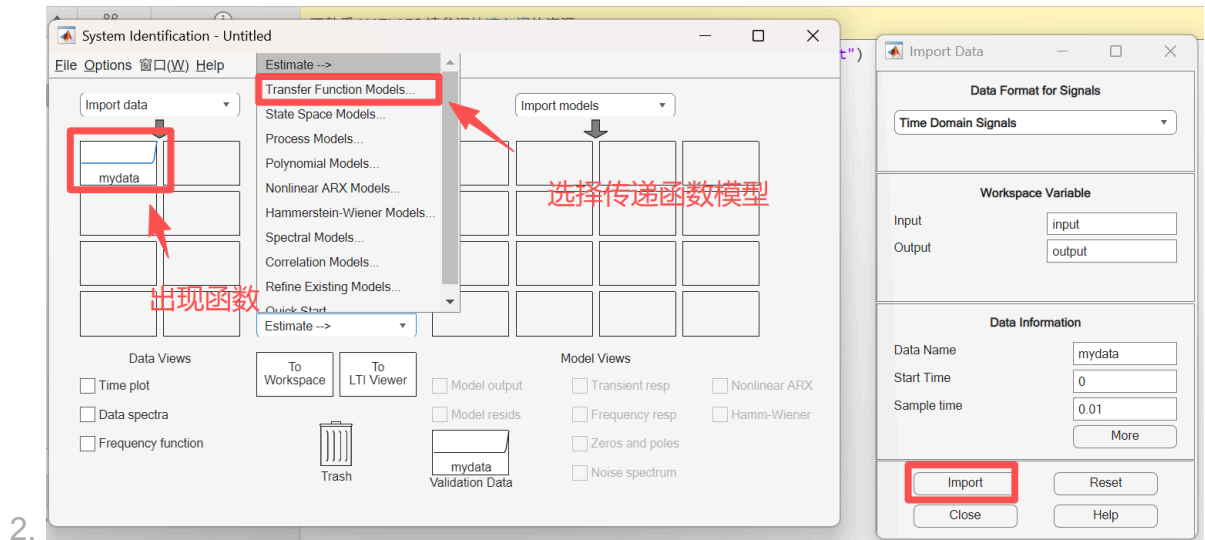
1. APP中选择System Identification (系统辨识)



2. 选择时域分析



- 注意，这里采样时间我的是10ms采样，填入了0.01，一般情况下要比10ms更快的采样，系统函数才足够准确



- 2.
3. 零极点选择。零点选择1个，极点选择2个。因为电机是二阶系统。[为什么电机是二阶系统请看此链接](#)

 Estimate Transfer Functions—□×

Model Structure

Estimation Options

Model name

Orders and Domain

Number of poles

Number of zeros

☒ Continuous-time

☐ Discrete-time (0.01 seconds)

☐ Feedthrough

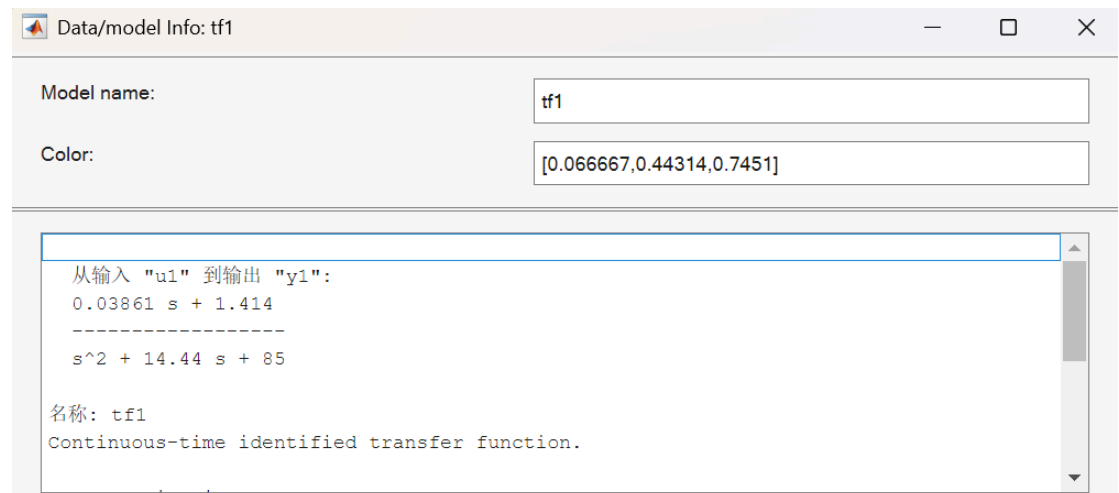
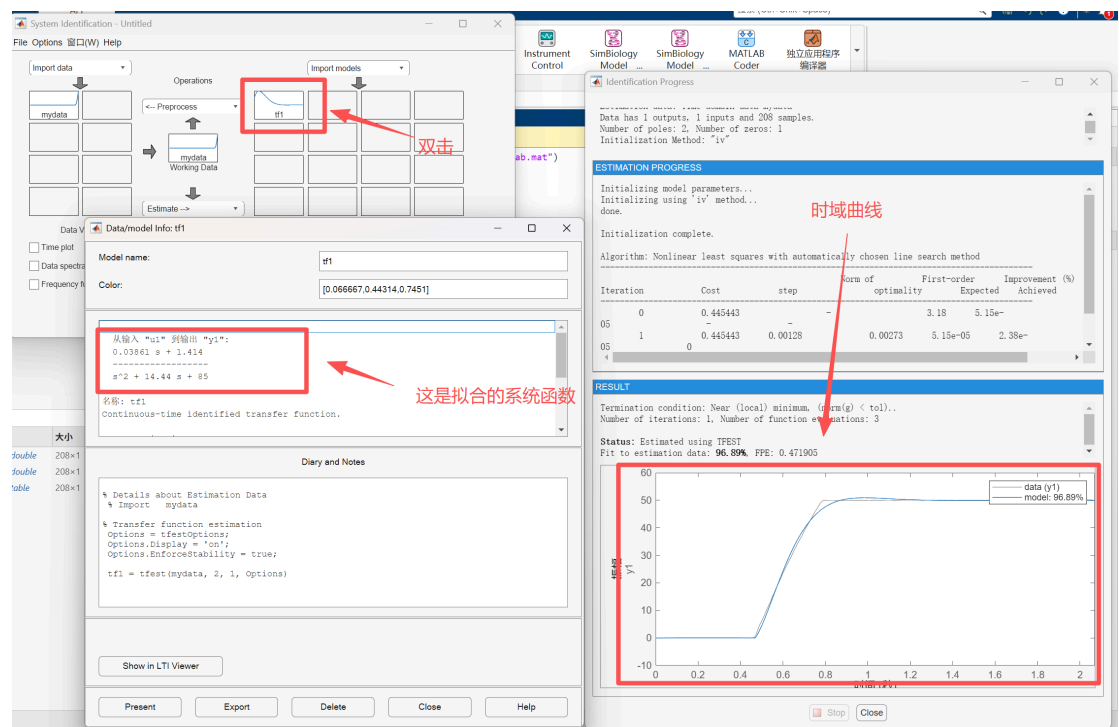
► Delay

帮助

Estimate

关闭

4. 拟合系统函数



2.4.3 Simulink

1. 修改simulink系统函数

1. 双击系统函数模块

2. 填入拟合系统函数的分子和分母

The screenshot shows a Simulink model of a control system. A step input is fed into a summing junction, which also receives feedback from the output. The output of the summing junction goes into a PID(z) controller block. The output of the PID controller goes into a Transfer Fcn block, which is labeled with the transfer function:
$$\frac{0.03944s + 1.337}{s^2 + 13.67s + 80.91}$$
. The output of the Transfer Fcn block is fed back to the summing junction. The parameter dialog for the Transfer Fcn block is open, showing the following settings:

- 分子系数: [0.03944 1.337]
- 分母系数: [1 13.67 80.91]
- 参数可调性: 自动
- 绝对容差: auto
- 状态名称: (例如, 'position')

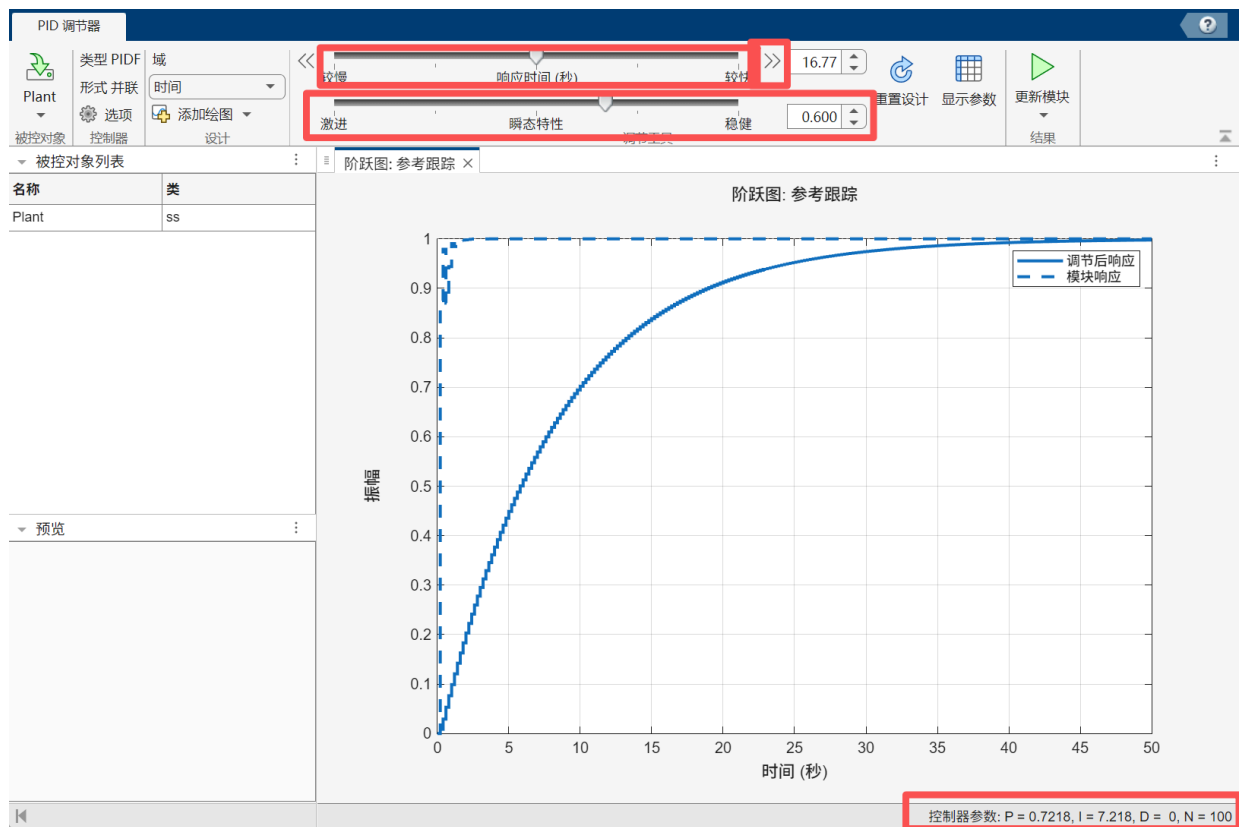
2. PID调节!

1. 控制器选择，可以选择PID、PI、PD等，根据实际情况进行选择，比如底盘电机一般为PI控制

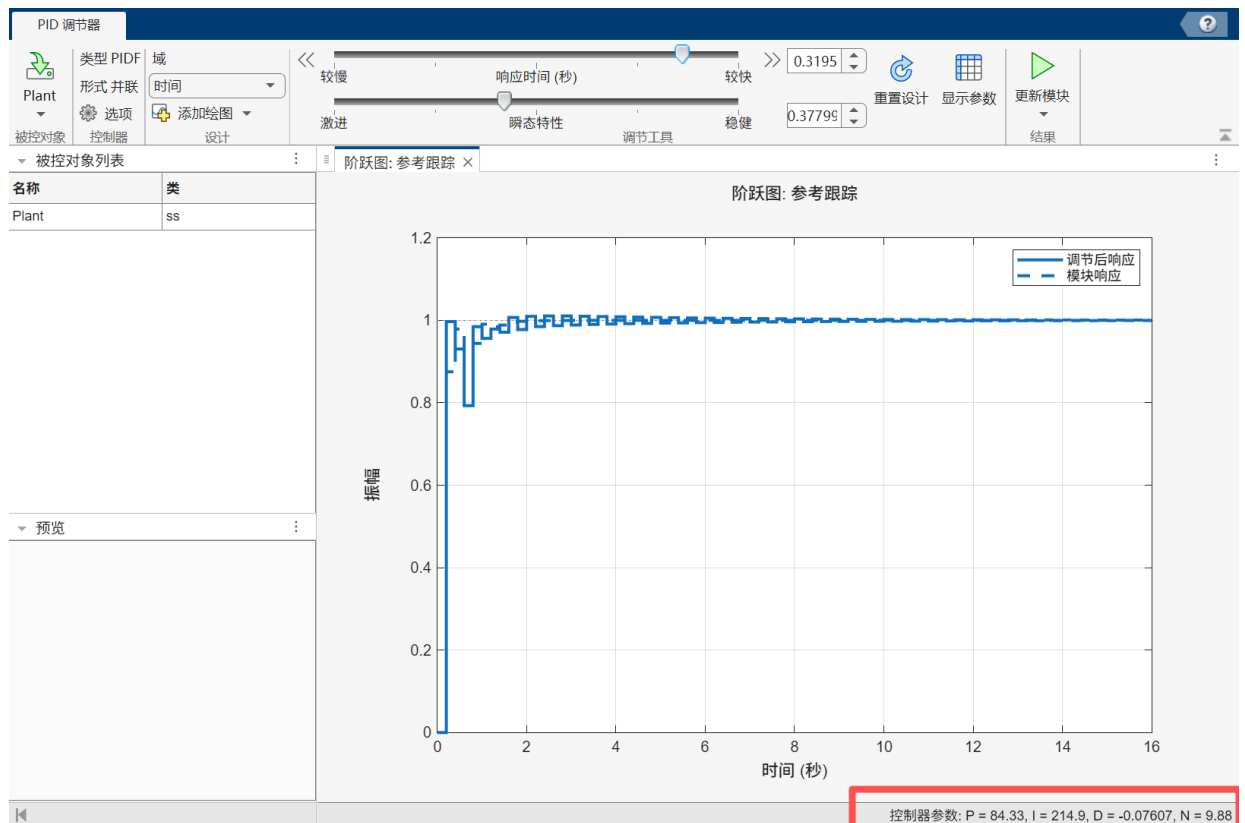
The screenshot shows a Simulink model of a control system. A step input is fed into a summing junction, which also receives feedback from the output. The output of the summing junction goes into a Discrete PID Controller block. The output of the Discrete PID Controller is fed back to the summing junction. The parameter dialog for the Discrete PID Controller is open, showing the following settings:

- 控制器: PID
- 形式: 并行
- 时域: ☒ 离散时间
- 离散时间设置: ☐ PID 控制器位于条件执行子系统内
- 采样时间(-1 表示继承): -1
- 积分器公式:
$$P + I \cdot T_s \frac{1}{z-1} + D \frac{N}{1 + N \cdot T_s \frac{1}{z-1}}$$
- 主要: 初始化 饱和 数据类型 状态属性
- 控制器参数: 源: 内部
- 比例(P): 72.9942019221052
- 积分(I): 219.576857708325 ☐ 使用 I*T_s (最适合 codegen)
- 微分(D): 0.101489516490627 ☐ 使用来自外部的导数
- 滤波器系数(N): 3.51759759628743 ☒ 使用滤波导数
- 自动调节: 选择调节方法: 基于传递函数(PID 调节器) ☒ 启用过零检测
- 调节...

2. 点击调节，出现调节曲线。这个时候响应速度很慢，通过拉取上面的响应时间和瞬态特征调节曲线。



3. 调节至满意的曲线，右下角会显示PID三个参数，此时 $P=84.33$ ， $I=214.9$ ， $D=-0.07607$ 。出现阶梯状曲线主要还是因为采样频率不够快。



4. 对比调好的底盘PI值，发现P项接近，I会有一定的偏差，但在可接受范围内，可提高采样频率，拟合的系统函数更符合电机特性！


```
void wheelChassis_Motor_Init(void)
{
    // 配置速度环PID参数（所有电机使用相同参数）
    // PID_Omega.Init(Kp, Ki, Kd, Kf, I_Max, Out_Max)
    for (int i = 0; i < 4; i++)
    {
        if ( i == 2)
        {
            G_Chassis_Motor[i].PID_Omega.Init(85.3f, 97.3f, 0.0f, 0.0f, 10000.0f, 12000.0f);
            continue;
        }
        else
        {
            G_Chassis_Motor[i].PID_Omega.Init(83.3f, 97.3f, 0.0f, 0.0f, 10000.0f, 12000.0f);
        }
    }

    // 初始化4个底盘电机对象（M3508 + C620）
    // CAN ID: 0x201-0x204, 控制模式：速度环
    G_Chassis_Motor[0].Init(&hcan1, CAN_Motor_ID_0x201, Control_Method_OMEGA, 19.0f, 20.0f);
    G_Chassis_Motor[1].Init(&hcan1, CAN_Motor_ID_0x202, Control_Method_OMEGA, 19.0f, 20.0f);
    G_Chassis_Motor[2].Init(&hcan1, CAN_Motor_ID_0x203, Control_Method_OMEGA, 19.0f, 20.0f);
    G_Chassis_Motor[3].Init(&hcan1, CAN_Motor_ID_0x204, Control_Method_OMEGA, 19.0f, 20.0f);
}
```